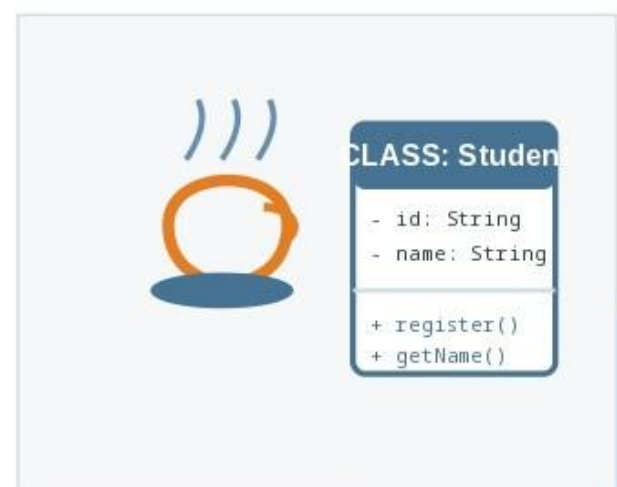


COMPLEX COMPUTING PROBLEM REPORT

SMART COURSE REGISTRATION &
STUDENT MANAGEMENT SYSTEM



GROUP MEMBERS:
ASADULLAH & HAFIZ MUHAMMAD SIDDIQ

SUBMITTED TO:
MUHAMMAD ANAS
DEPARTMENT OF COMPUTER SCIENCE

SMART COURSE REGISTRATION & STUDENT MANAGEMENT SYSTEM
(SCRNS)

TABLE OF CONTENTS

1. Introduction & System Overview	02
2. Software Requirements Specification (SRS)	03
3. Technology Stack & Architecture Rationale	04
4. Database Design & Low-Level Data Storage	05
5. System Modules & Operational Flow	06
6. Implementation Details & Configurations	07
7. Testing, Verification, and Future Expansion	08

INTRODUCTION & SYSTEM OVERVIEW

SMART COURSE REGISTRATION & STUDENT MANAGEMENT SYSTEM (SCRNS)

1.1 Project Background

Educational institutions require robust administrative tools to handle student registration and course enrollment. The Smart Course Registration & Student Management System (SCRNS) is an optimized, lightweight desktop application built on Java that transitions administrative pipelines from legacy spreadsheets into a unified, database-driven environment.

1.2 Problem Statement & Objectives

Traditional desktop applications often suffer from high resource demands, complex external SQL server configurations, and outdated UI layouts. SCRNS addresses these challenges via:

- **Zero-Configuration Deployment:** Uses an embedded database engine to eliminate complex installations.
- **Modern Visuals:** Implements a clean, hardware-accelerated dark theme using FlatLaf.
- **Strict Architecture Separation:** Employs proven design patterns to eliminate monolithic, unmaintainable code structures.

1.3 Scope of the System

The system encompasses target operational boundaries designed to optimize administration pipelines:

- Comprehensive CRUD (Create, Read, Update, Delete) student administration pipelines.
- State-synchronized graphical components for real-time visual feedback.
- Hybrid data management implementing relational SQLite storage paired seamlessly with binary file serialization.

COMPLEX COMPUTING PROBLEM REPORT

SMART COURSE REGISTRATION & STUDENT MANAGEMENT SYSTEM

2. SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

2.1 Functional Requirements

REQUIREMENT ID	DESCRIPTION
FR-1 (Registration)	Validates and inserts unique alphanumeric IDs and student names.
FR-2 (Record Retrieval)	Instantly queries and displays active records in a tabular format on startup.
FR-3 (Interactive Forms)	Clicking a table record auto-populates editing forms.
FR-4 (Record Upgradation)	Allows modification of existing registered students.
FR-5 (Record Elimination)	Safely deletes records from the primary database.
FR-6 (Real-time Filtering)	Features a dynamic search bar to filter student records instantly.
FR-7 (Hybrid State Saving)	Serializes nested course objects directly to a local flat file.

2.2 Non-Functional Requirements & Constraints

REQUIREMENT ID	SPECIFICATION / TARGET METRIC
NFR-1 (Performance)	UI transitions and database transactions must execute within $t \leq 200\text{ ms}$.
NFR-2 (Aesthetics)	Modern, highly legible dark theme for optimized administration workflow.
NFR-3 (Portability)	Zero-installation footprint; runs on any OS with a JVM (Java Virtual Machine).
NFR-4 (Data Integrity)	Strict Primary Key constraints enforced on Student IDs.

COMPLEX COMPUTING PROBLEM REPORT

SMART COURSE REGISTRATION & STUDENT MANAGEMENT SYSTEM

3. TECHNOLOGY STACK & ARCHITECTURE

3.1 Technology Evaluation Table

COMPONENT / LAYER	TECHNOLOGY CHOSEN	TECHNICAL RATIONALE & BENEFITS
Core Platform	Java (JDK 26)	Cross-platform compatibility and robust memory management.
GUI Framework	Java Swing	Stable, native library for seamless event and window bindings.
Look & Feel	FlatLaf (v3.6)	Sophisticated dark theme; solves high-DPI scaling issues.
Database Engine	SQLite (JDBC v3.50.3.0)	Embedded engine; zero manual server installation required.
Build Tool	Maven	Standardizes dependencies and builds production JARs.
Backup Persistence	Java Serialization	Handles complex binary saving for nested non-relational objects.

3.2 Architectural Patterns (MVC & DAO)

SCRNS separates code into distinct operational layers to prevent code spaghetti:

- **Model Layer** (`com.scrns.model`) :
Pure POJOs (e.g., `Student.java`) representing data blueprints.
- **Service Layer** (`com.scrns.service`) :
Validates formatting rules and coordinates data flow.
- **Data Access Layer** (`com.scrns.database`) :
Uses the Data Access Object (DAO) pattern to isolate raw SQL queries from the UI, allowing future database migrations without modifying the visual front-end.
- **Presentation Layer** (`com.scrns.gui`) :
Handles visual rendering and captures user input events.

COMPLEX COMPUTING PROBLEM REPORT

SMART COURSE REGISTRATION & STUDENT MANAGEMENT SYSTEM

4. DATABASE DESIGN & LOW-LEVEL STORAGE

4.1 Relational Schema Design (SQLite)

Relational data is contained locally within the `scrns.db` file. Uniqueness and data integrity are enforced directly at the schema layer through strict relational rules:

```
CREATE TABLE students (  
  id TEXT PRIMARY KEY,  
  name TEXT NOT NULL  
);
```

4.2 Runtime Diagnostic Analysis

Active transaction tests verify successful indexing via `sqlite_autoindex_students_1` on the following primary persistent records:

- **IU09-0325-0046** → Hafiz Muhammad Siddiq (Active primary record)
- **IU09-0325-0049** → Hafi (Partial tracking record)
- **001** → asadullah (Standard validation record)

4.3 Alternative State Serialization (course.dat)

To bypass heavy join-table overhead during rapid prototyping, non-relational nested structures are handled using direct Java Object Serialization:

Target File:	<code>course.dat</code>
Class Blueprint:	<code>com.scrns.model.Course</code>
Fields Wrapped:	<code>courseName (String), students (ArrayList)</code>

COMPLEX COMPUTING PROBLEM REPORT

SMART COURSE REGISTRATION & STUDENT MANAGEMENT SYSTEM

6. IMPLEMENTATION DETAILS (CONFIGURATIONS)

6.1 Dependency Management (pom.xml)

Clean Maven project object model configuration utilizing structured build dependencies to automate external library management:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/
<modelVersion>4.0.0</modelVersion>
<groupId>org.example</groupId>
<artifactId>SCRNS</artifactId>
<version>1.0-SNAPSHOT</version>

<properties>
  <maven.compiler.source>26</maven.compiler.source>
  <maven.compiler.target>26</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <!-- FlatLaf Look and Feel Library -->
  <dependency>
    <groupId>com.formdev</groupId>
    <artifactId>flatlaf</artifactId>
    <version>3.6</version>
  </dependency>

  <!-- SQLite Database JDBC Driver -->
  <dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.50.3.0</version>
  </dependency>
</dependencies>
</project>
```

6.2 Workspace Hygiene (.gitignore)

Excludes temporary environment-specific configurations and build output artifacts from version control tracking to prevent workspace clutter:

- **Build Outputs:** `target/` , `/nbbuild/` , `build/`
- **IDE Configs:** `.idea/` , `.vscode/` , `SCRNS.iml`
- **OS Cache:** `.DS_Store`

COMPLEX COMPUTING PROBLEM REPORT

SMART COURSE REGISTRATION & STUDENT MANAGEMENT SYSTEM

5. SYSTEM MODULES & OPERATIONAL FLOW

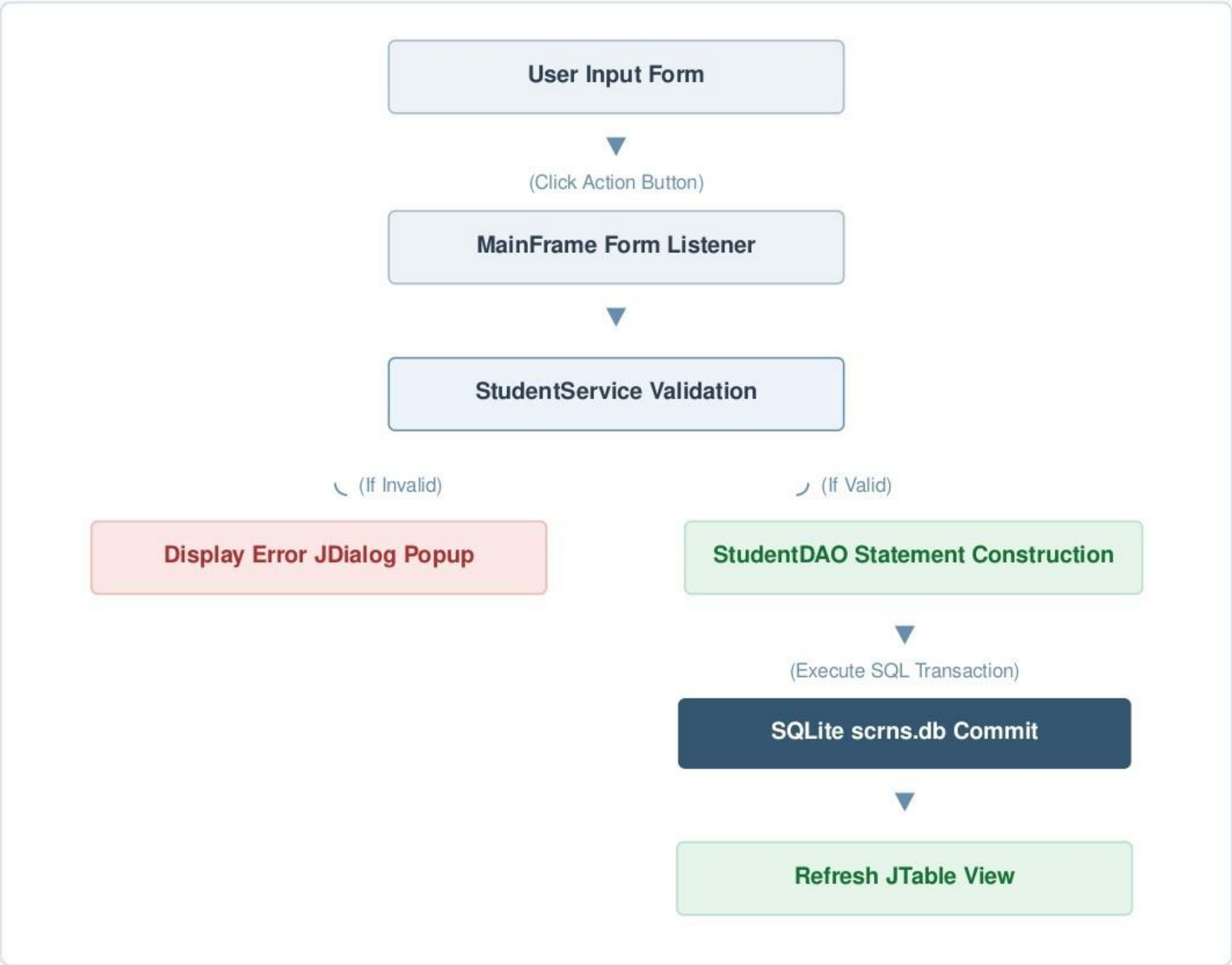
5.1 Presentation Layout

Powered by FlatLaf 3.6, the user interface is programmatically split into two primary operational panels to enhance administrator efficiency:

- 1. **Left-side Entry Form:** Houses interactive data text fields (Student ID, Student Name) aligned alongside explicit workflow Action buttons (Create, Update, Delete, Reset).
- 2. **Right-side Data Panel:** Integrates an active, real-time search filter directly linked to a `TableRowSorter`, overlaying a highly responsive and synchronized `JTable` component.

5.2 Business Logic & CRUD Lifecycle Workflow

Before any low-level database execution takes place, the architectural Service Layer intercepts the payload to trim trailing whitespaces and strictly reject empty or formatting-violating submissions:



COMPLEX COMPUTING PROBLEM REPORT

SMART COURSE REGISTRATION & STUDENT MANAGEMENT SYSTEM

7. TESTING, SCALABILITY & CONCLUSION

7.1 Testing Scenarios & Validations

- **Duplicate ID Test:** Inserting an existing Primary Key successfully triggers an index constraint violation, safely caught by the DAO and alerted via the UI.
- **Form Emptiness Test:** Intercepted immediately by the Service layer, blocking bad database updates.
- **Resource Leaks:** Handled via standard try-with-resources blocks to ensure SQLite connection locks release cleanly on shutdown.

7.2 Scalability Roadmap

1. **Role-Based Access Control (RBAC):** Integrate cryptographic credential hashing to separate admin tasks from standard student views.
2. **Reporting APIs:** Incorporate Apache POI / JasperReports for automated Excel exports and PDF registry sheets.
3. **Cloud Migration:** Transition to a distributed RESTful API backend (Spring Boot), decoupled Web client frontend (React/Angular), and enterprise PostgreSQL cloud storage.

7.3 CONCLUSION

SCRNS successfully coordinates a modern FlatLaf UI, stable DAO design patterns, and an embedded SQLite relational database. The resultant application features a highly modular, decoupled architecture that offers an efficient baseline ready for enterprise web adaptation.

OUTPUT DETAILS

- *4 tabs: Students, Courses, Enrollment, My Courses*
- *Student records: 5 students stored (Asad Ullah, Siddiq, Maqsood, Khizer, Raza)*
- *CRUD operations: Add, Search, Update, Delete, Load All*
- *Data display: Table view with Student ID and Student Name columns*
- *UI style: Dark theme desktop application (likely built in Java)*
- *Purpose: To enroll students into courses and manage their academic records*

SCRNS - Student Course Registration System

Students

Courses

Enrollment

My Courses

Student ID

Student Name

Search

Add

Update

Delete

Load All

Student ID	Student Name
1	Asad Ullah
2	Siddiq
3	Maqsood
4	khizer
5	Raza

OUTPUT DETAILS

- 5 courses available: *OOP Lab, OOP Theory, Multivariate Calculus, Digital Logic Design, Data Structure*
- Each course has a capacity of 40 students
- Table displays 4 columns: *Course ID, Course Name, Capacity, Enrolled Students*
- Operations available: *Add Course, Delete Course, Load All*
- Input fields: *Course ID, Course Name, Course Capacity*
- Enrollment status: *OOP Lab has 2 students, Digital Logic Design and Data Structure have 1 each, rest have 0*

SCRNS - Student Course Registration System

Students

Courses

Enrollment

My Courses

Student ID

Search

Student Name

Add

Update

Delete

Load All

Student ID	Student Name
1	Asad Ullah
2	Siddiq
3	Maqsood
4	khizer
5	Raza

OUTPUT DETAILS

- *Enrollment tab allows linking a student to a specific course using Course ID and Student ID*
- *Three operations available: Enroll, Unenroll, and View Students*
- *Input fields: Course ID and Student ID to perform enrollment actions*
- *Table displays enrolled students with Student ID and Student Name columns*
- *View Students button loads all students enrolled in the entered Course ID*
- *Currently the table is empty, meaning no course has been selected yet*

SCRNS - Student Course Registration System

Students

Courses

Enrollment

My Courses

Course ID

Enroll

Student ID

Unenroll

View Students

Student ID	Student Name
------------	--------------

OUTPUT DETAILS

- *Course ID 001 (OOP Lab) is selected and View Students button was clicked to load enrolled students*
- *2 students are currently enrolled in this course: Asad Ullah (ID 1) and Maqsood (ID 3)*
- *This confirms the enrolled count of 2 shown earlier in the Courses tab for OOP Lab*
- *Student ID field is empty, meaning only viewing is being done, not enrolling or unenrolling*
- *The table shows Student ID and Student Name of all enrolled students in the selected course*
- *This screen proves that the enrollment linking between students and courses is working correctly*

SCRNS - Student Course Registration System

Students

Courses

Enrollment

My Courses

Course ID

001

Enroll

Student ID

Unenroll

View Students

Student ID	Student Name
1	Asad Ullah
3	Maqsood

OUTPUT DETAILS

- *My Courses tab allows a student to search all courses they are enrolled in using their Student ID*
- *Student ID 1 (Asad Ullah) was searched using the Find Courses button*
- *Result shows 5 courses: OOP Lab, OOP Theory, Multivariate Calculus, Digital Logic Design, and Data Structure*
- *Table displays Course ID, Course Name, and Capacity for each enrolled course*
- *Each course has a capacity of 40 students, confirming consistent data across the system*
- *This tab acts as a personal course viewer for individual students to track their enrollments*

SCRNS - Student Course Registration System

Students

Courses

Enrollment

My Courses

Student ID:

1

Find Courses

Course ID	Course Name	Capacity
001	OOP LAB	40
002	OOP THEORY	40
003	MULTIVARIATE CALCULUS	40
004	Digital Logic Design	40
005	Data Structure	40